

The Complete Guide to Rate Limiting

Five algorithms, drawn rather than described — then what changes the moment the limiter runs on more than one machine, and what it takes to run one in production.

Five algorithms, one picture each

Fixed window, sliding log, sliding counter, token bucket, leaky bucket.

One trace, five verdicts

The same 23 requests through all five, simulated — not illustrated.

Distributed, honestly

Local vs central vs gossip, the read-modify-write race, and the atomic fix.

Production-ready, not textbook

Fail open or closed, picking the number, and the eight easy mistakes.

What's inside

Built to be read by looking. Every algorithm gets one page and one diagram; Part 4 puts all five side by side on the same traffic, and Part 5 is where interviews are actually won or lost.

1 What a limit is, and where it goes

Rate limit vs throttle vs quota vs load shedding, and the layer to put each one on.

2 The window algorithms

Fixed window and its famous boundary bug; sliding window log; sliding window counter.

3 The bucket algorithms

Token bucket, leaky bucket in both forms, and why two of them are the same algorithm.

4 All five, side by side

One arrival trace run through every algorithm, plus the decision in four questions.

5 Rate limiting across many machines

Local vs central vs gossip, the counter race, Lua atomicity, and Redis on the hot path.

6 Production concerns

Fail open or closed, picking the number, and the eight mistakes that bite in prod.

7 The interview itself

How to structure the answer, 14 asked-in-real-interviews Q&A, and the red flags.

★ One-page cheat sheet

The night-before page: the five algorithms in one table, the numbers, eight facts.

Part of the HLD concept series. Redis mechanics, caching and resiliency patterns are separate volumes; this one is only about deciding whether a request may proceed.

How to use this guide

Look first

Each algorithm is one page: a diagram, ten lines of code, and the trade in two sentences. Read the picture, then the words.

Say out loud

The coloured callouts are the exact sentences that earn the point in an interview.

Defend

Every algorithm here is wrong for something. Knowing which is the answer.

FOUR WORDS INTERVIEWERS USE INTERCHANGEABLY — YOU SHOULD NOT

Rate limiting caps requests per unit of time to protect the system. **Throttling** is what you do to a caller that exceeds it — reject, delay or degrade. **A quota** is a business allowance over a long period (10,000 calls a month) and is billing, not protection. **Load shedding** drops traffic when the system is already in trouble, regardless of who sent it. A serious design has all four, and they fail differently.

Fixed window
count, reset on the clock



2x across the boundary

Sliding log
remember every timestamp



exact, and expensive

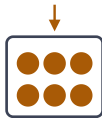
Sliding counter
blend two counts

80

20

$20 + 80 \times 0.7 = 76$
two ints, ~99.9% right

Token bucket
spend a token



bursts up to capacity

Leaky bucket
is there room?



the same thing

Two of these five are the same algorithm. One of them is broken. Knowing which is the whole topic.

Figure 0.1 — The whole book on one row. Every page that follows is one of these five boxes drawn larger.

THE ONE THING TO INTERNALISE

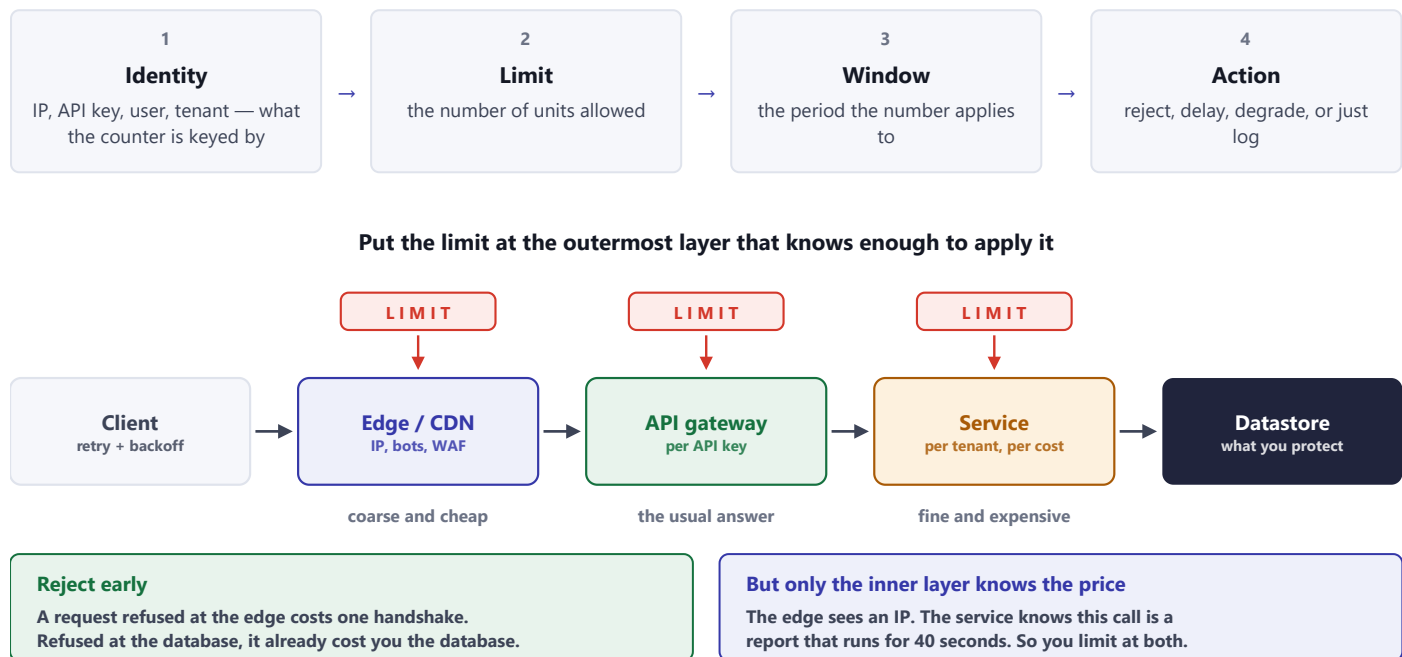
Every algorithm on the next six pages answers the same question — “**has this caller used more than X in the last Y?**” — and they differ only in how much memory they spend to answer it and how honest the answer is at the edges. Pick the cheapest one whose dishonesty you can live with.

What a limit is, and where it goes

PART 1

Before any algorithm: a rate limit is four decisions — who you count, how many, over how long, and what you do when the answer is no.

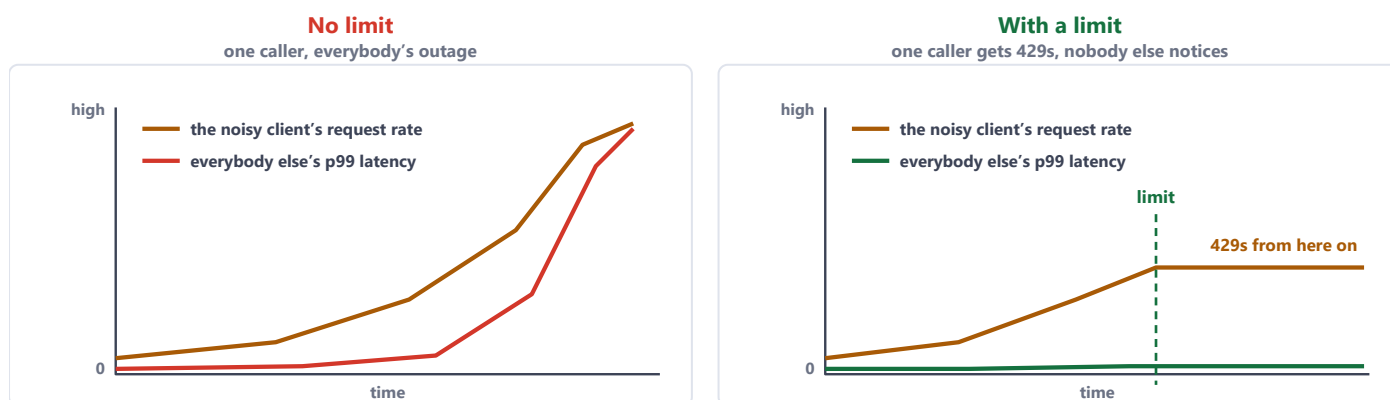
1.1 The four inputs



Rate limiting protects the system. Quotas bill the customer. Load shedding saves you when both have already failed.

Figure 1.1 — The same request can meet three different limiters. Each one knows something the others do not.

1.2 Why you are doing this at all



The limit does not make the noisy client fast. It makes everybody else unaffected — and turns a mystery into a 429.

Figure 1.2 — *What the limit is actually for: not punishing the noisy caller, but disconnecting everybody else from them.*

Protect capacity

One client should not be able to consume the headroom you sized for everyone.

Contain abuse

Credential stuffing, scraping and enumeration are all rate problems before they are anything else.

Control cost

When a request costs you money — an LLM call, an SMS, a third-party API — the limit is the budget.

Enforce fairness

A shared pool with no per-tenant limit is a pool one tenant will take.

Meet someone else's limit

You are often the client. Shaping your own outbound traffic is the same problem from the other side.

Make failure predictable

A limit turns “everything got slow” into “that one caller got 429s”, which is a far better outage.

INTERVIEW LINE

“I would start by asking what the limit is protecting. If it is capacity, I limit at the gateway on the API key. If it is cost, the limit belongs in the service and it counts cost units, not requests. If it is abuse, it is per IP at the edge and it is deliberately blunt. Those are three different limiters and I would probably ship all three.”

The window algorithms

PART 2

Chop time into windows and count. Three variants, in the order interviewers walk you through them: the cheap one, the exact one, and the one you actually run.

2.1 Fixed window counter

One counter, one window start. When the clock crosses a window boundary the counter resets. That is the whole algorithm.



One counter, one timestamp. INCR + EXPIRE in Redis is the whole implementation.

That cheapness is the only reason anyone accepts the bug on the next page.

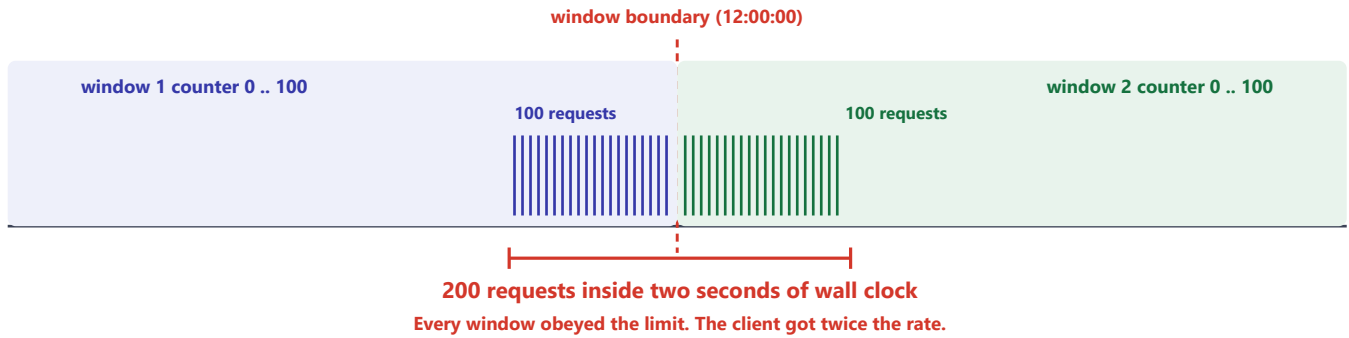
Figure 2.1 — Three consecutive windows, limit five. The counter drops to zero at every boundary, regardless of what happened one millisecond earlier.

Fixed window — the whole thing

```
// roll forward in whole windows so the grid stays aligned
if (now - windowStart >= windowMillis) {
    windowStart += ((now - windowStart) / windowMillis) * windowMillis;
    count = 0;
}
if (count < maxRequests) { count++; return true; }
return false;
```

O(1) memory, O(1) time. In Redis this is literally **INCR key** then **EXPIRE key window** — two commands, no script.

2.2 ...and the bug that spawned three more algorithms



This single defect is the reason the other four algorithms exist.
Sliding window anchors the window to "now" instead of to the clock. Buckets never reset at all.

Figure 2.2 — *The boundary burst. With a limit of 100 per minute, a client can legally take 200 in a two-second span straddling the reset.*

When it is fine

Internal traffic, a crude abuse ceiling, or any limit set so far above real usage that 2x still does not hurt.

When it is not

Public APIs, anything adversarial, and anything you bill for — a customer who notices will call the 2x a bug, and they will be right.

Fixed window in Redis — and the second bug hiding in it

```
MULTI
  INCR ratelimit:{user42}:1724500860 -- key carries the window id
  EXPIRE ratelimit:{user42}:1724500860 60
EXEC
```

-- EXPIRE on every call resets the TTL, so a busy key never dies.
-- Set it only when INCR returns 1, or SET key 0 EX 60 NX first.

The window id is *in the key*, so the old window expires on its own and there is no reset code at all.

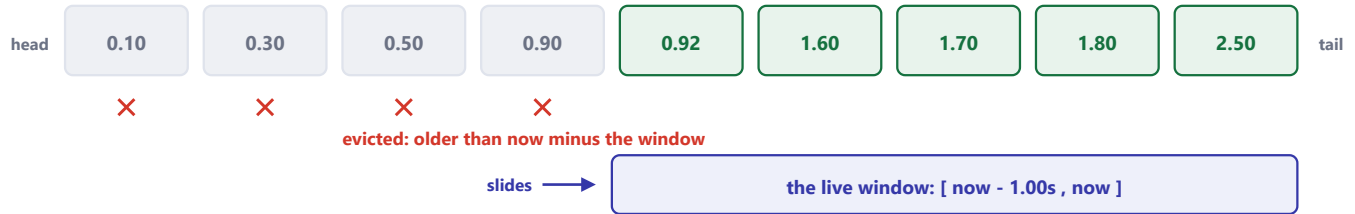
HOW TO USE THIS IN AN INTERVIEW

Implement fixed window first, then attack it yourself before the interviewer does: **"this is $O(1)$ and two Redis commands, but it lets a client take twice the limit across a boundary — so if this is a public API I would move to a sliding window."** Volunteering the flaw is worth more than avoiding it.

2.3 Sliding window log — the exact one

Keep a timestamp for every allowed request. Drop the ones that have aged out, count what is left. No approximation anywhere.

The window is anchored to now, so there is no boundary to exploit



What you get

Exactly N per window. No approximation, no boundary case, no argument with the customer about what the number means. Rejected requests are not recorded, so a client being throttled cannot deepen its own hole.

What it costs

One timestamp per allowed request, per client.
1M clients x 1,000 req/window x 8 bytes = 8 GB
of RAM that does nothing but remember.
Eviction is amortised O(1), but the memory is the reason this rarely ships as-is at scale.

Figure 2.3 — The window is a bracket anchored to now, sliding continuously. There is no boundary, so there is nothing to exploit.

Sliding window log

```
long cutoff = now - windowMillis;
// evict what slid out
while (!log.isEmpty()
    && log.peekFirst() <= cutoff)
    log.pollFirst();

if (log.size() >= maxRequests)
    return false;
log.offerLast(now);
return true;
```

The cost, in numbers

per client : up to LIMIT timestamps
per timestamp: 8 bytes (+ overhead)

1M clients
x 1,000 req/window
x 8 bytes = 8 GB

// and that is before Redis
// ZSET key overhead

Amortised **O(1)** per call — each timestamp is added once and evicted once — but the memory is why this rarely ships unmodified.

2.4 Sliding window counter — the one you actually run

Throw the timestamps away. Keep two fixed-window counts and interpolate between them.

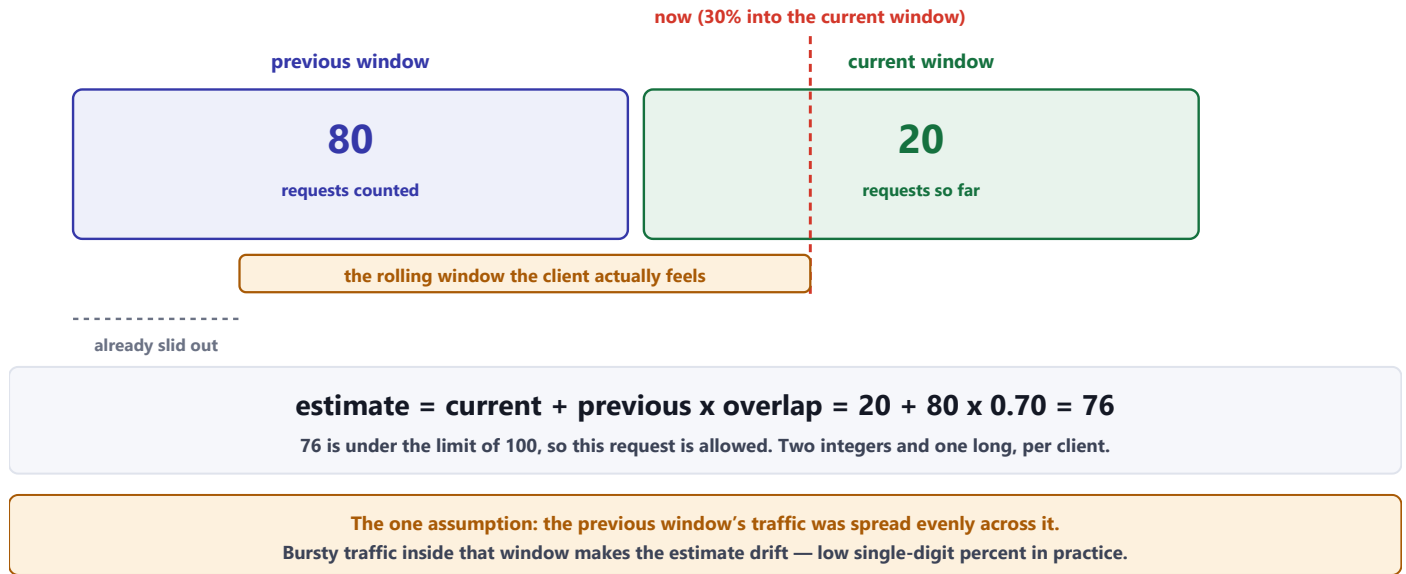


Figure 2.4 — Two counters and one multiply. The previous window's contribution decays smoothly to zero, which is exactly what kills the boundary burst.

Sliding window counter

```
long b = now / windowMillis;
if (b == cur + 1) { prev = curr; curr = 0; cur = b; } // rolled once
else if (b > cur + 1) { prev = 0; curr = 0; cur = b; } // idle client

double overlap = 1.0 - (now - cur * windowMillis) / (double) windowMillis;
double estimate = curr + prev * overlap;

if (estimate < maxRequests) { curr++; return true; }
return false;
```

Two ints and a long per client. Cloudflare reported roughly 0.003% error against an exact log on real traffic — and this is the algorithm most large public APIs are actually running.

The bucket algorithms

PART 3

Stop thinking in windows. A bucket has no boundary to reset at, which is why these two never had the fixed-window bug in the first place.

3.1 Token bucket



Why it is the default answer

Two numbers per client, $O(1)$ per call, and it lets a quiet client spend a saved-up burst — which real traffic does.

The two dials, said correctly

capacity sets how big a burst you tolerate; refill rate sets the sustained rate. They are independent knobs.

Figure 3.1 — Tokens accrue while you are quiet and are spent when you are busy. Capacity is the burst you tolerate; refill rate is the rate you actually enforce.

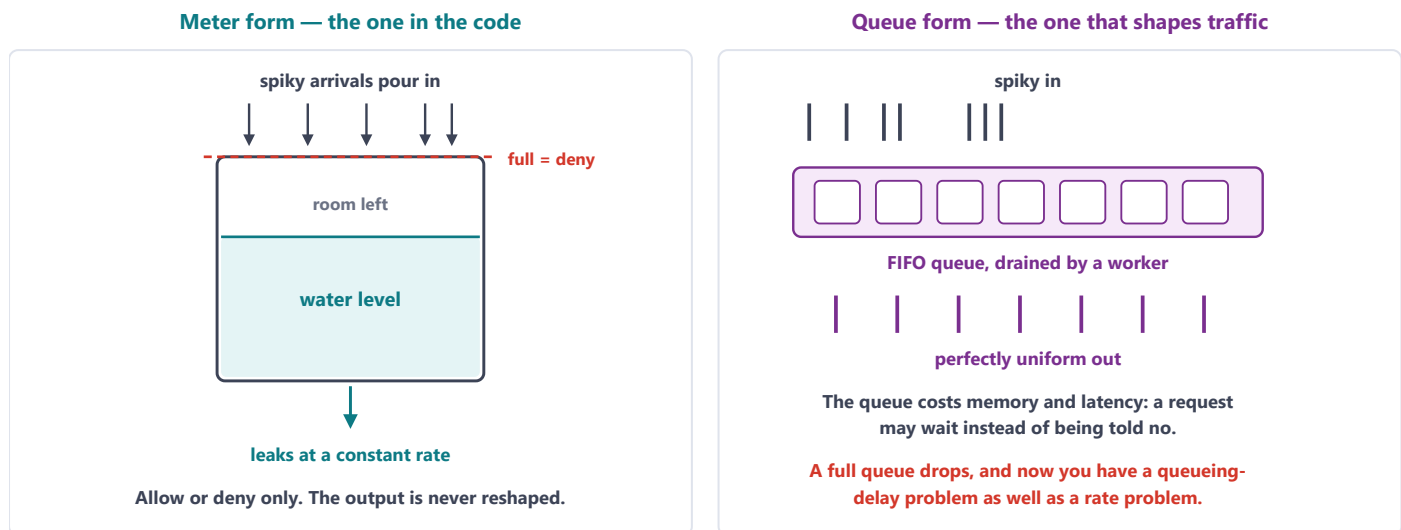
Token bucket — lazy refill, no background thread

```
// how many tokens WOULD have arrived since the last call
double add = (now - lastRefillNanos) / 1e9 * refillRatePerSecond;
tokens = Math.min(capacity, tokens + add); // the cap is the point
lastRefillNanos = now;
```

```
if (tokens >= 1.0) { tokens -= 1.0; return true; }
return false;
```

Never spawn a timer. Computing the refill on read is mathematically identical to a continuous top-up and costs nothing. Saying this unprompted is a strong signal.

3.2 Leaky bucket — and the answer to “aren’t these the same?”



Say this when asked: the meter form of leaky bucket and the token bucket are the same algorithm inside out. “Is there room?” and “do I have a token?” are the same comparison. Only the queue form behaves differently.

Figure 3.2 — Two different things share one name. The meter is a limiter; the queue is a traffic shaper, and it is the only one of the five that can make a request wait.

Leaky bucket, meter form

```
double out = (now - lastLeakNanos)
              / 1e9 * leakRatePerSecond;
water = Math.max(0.0, water - out);
lastLeakNanos = now;

if (water + 1.0 <= capacity) {
    water += 1.0; return true;
}
return false;
```

Compare it line by line with token bucket. **water = capacity - tokens**. Same compare, opposite sign.

The two forms, decided

```
meter -> allow / deny
         0(1) memory
         == token bucket

queue  -> allow / wait / drop
         0(queue) memory
         adds latency
         smooths the OUTPUT
```

Reach for the queue form only when something downstream genuinely needs a steady feed — a partner API that permits 10 calls a second.

All five, side by side

PART 4 · THE PAGE TO MEMORISE

The five algorithms are not opinions. Run them against the same arrivals and they disagree in specific, predictable ways.

One arrival trace, 23 requests, limit = 5 per second

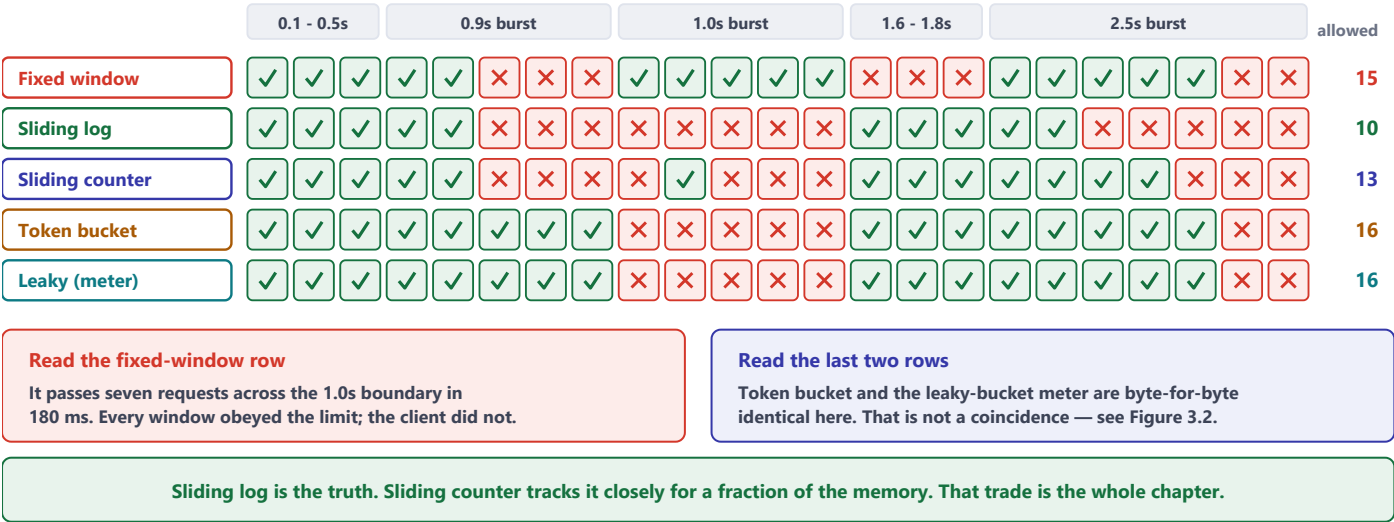


Figure 4.1 — Twenty-three requests, limit five per second, every verdict produced by running the reference implementations — not drawn by hand.

Fixed window let 15 through — and broke the rule

Seven of them inside 180 ms across the boundary. It is the only row here that violates the limit it was configured with.

Sliding log let 10 through — the truth

Never more than five in any one-second span, anywhere in the trace. This row is the definition the others are approximating.

Sliding counter let 13 through — close, and cheap

It under-counts where the previous window was back-loaded, because it assumes an even spread. Two integers bought that.

Both buckets let 16 through — identically

They start with a full burst allowance and refill continuously, so they are more permissive by design, not by accident.

4.2 The comparison table

Algorithm	Memory / client	Bursts	Accuracy	Reach for it when
Fixed window	1 counter + 1 timestamp	2x at the boundary	Poor at edges	It is internal, or the limit is generous enough that 2x is harmless
Sliding log	Up to LIMIT timestamps	None — exactly N per window	Exact	The number is billable, or the client count is small
Sliding counter	2 ints + 1 long	Smoothed away	~99.9%	Public API, millions of clients — the production default
Token bucket	1 double + 1 timestamp	Up to capacity, by design	Exact for its own rule	Human-paced traffic that is legitimately spiky
Leaky bucket (meter)	1 double + 1 timestamp	Up to capacity	Same as token bucket	You prefer the “is there room?” framing
Leaky bucket (queue)	The queue itself	Absorbed, then delayed	Output is perfectly uniform	Something downstream needs a steady feed and can wait

Pick from the requirement, not from the algorithm

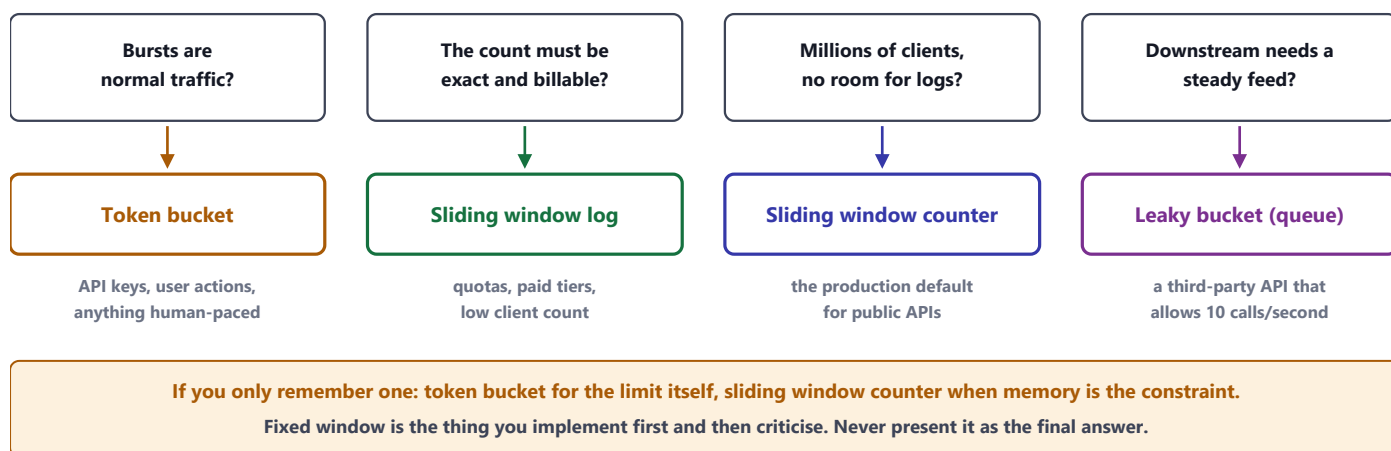


Figure 4.2 — Four questions, four answers. Any of them is defensible; picking without asking one of them is not.

Rate limiting across many machines

PART 5 · WHERE INTERVIEWS ARE WON

Every algorithm so far assumed one process and one lock. Fifty app servers behind a load balancer break that assumption immediately, and this is the follow-up that separates candidates.

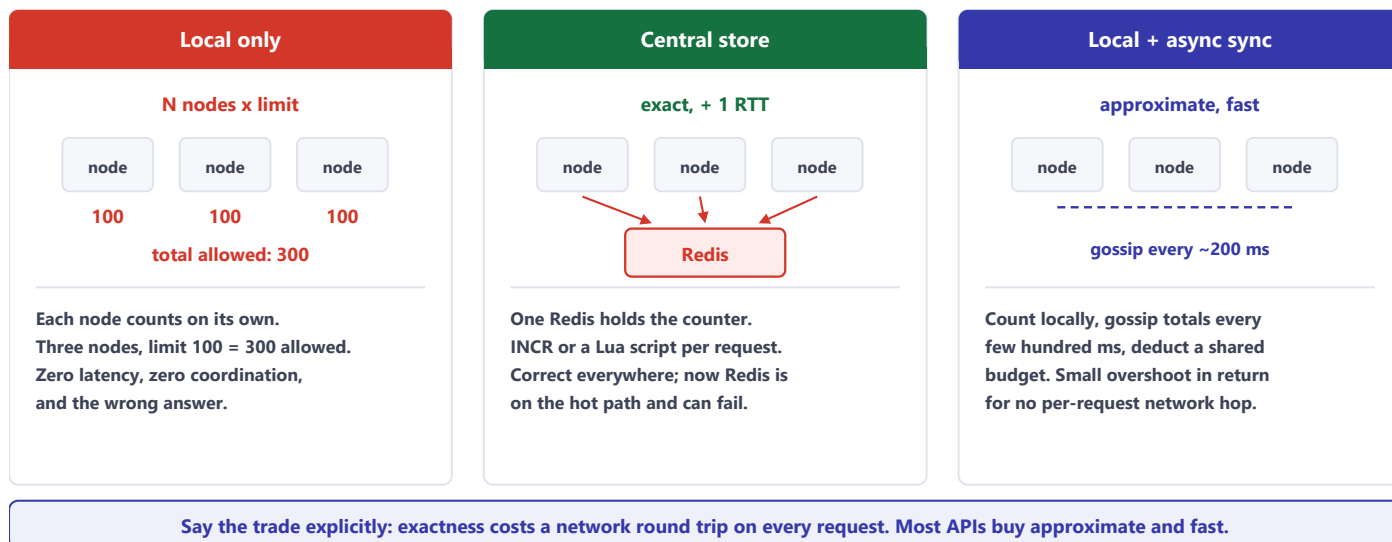


Figure 5.1 — The three topologies. There is no free option: you are choosing which of exactness, latency and availability to give up.

INTERVIEW LINE

“With fifty nodes, a purely local limiter enforces fifty times the limit — so the state has to be shared. I would keep it in Redis with a Lua script so the whole decision is one atomic round trip. If that round trip is too expensive at our request rate, I would move to local counters with an asynchronous sync and say out loud that the enforced limit is now approximate by a few percent.”

5.2 The race, and why one atomic operation fixes it

Two nodes, one counter at 99, limit 100



counter ends at 101. One request over the limit, silently.

Fix 1 — one atomic op

INCR key ; EXPIRE key 60

The read and the write are the same command, so there is no window to interleave into.

Works for fixed window. Not enough for the rest.

Fix 2 — a Lua script, for every other algorithm

Token bucket needs read tokens → compute refill → compare → write back. Four steps, one client, no lock.

Redis runs a Lua script atomically on one shard, so the whole decision is one round trip and one critical section.

Keep every key for one client on one shard — a hash tag, or the client id as the whole key.

A script that touches two shards will not run, and a counter split across shards is not a counter.

Figure 5.2 — Read-modify-write across two nodes. The counter is shared, but the decision was not — so both nodes decided yes.

Distributed token bucket — one Lua script, one round trip, atomic

```
-- KEYS[1] = bucket:{clientId}  ARGV = now, rate, capacity
local b  = redis.call('HMGET', KEYS[1], 'tokens', 'ts')
local tk = tonumber(b[1]) or tonumber(ARGV[3])
local ts = tonumber(b[2]) or tonumber(ARGV[1])
tk = math.min(tonumber(ARGV[3]), tk + (ARGV[1] - ts) * ARGV[2])
local ok = tk >= 1
if ok then tk = tk - 1 end
redis.call('HSET', KEYS[1], 'tokens', tk, 'ts', ARGV[1])
redis.call('EXPIRE', KEYS[1], 60)
return { ok and 1 or 0, tk }
```

The braces matter. `bucket:{user42}` forces every key for one client onto one Redis slot — a script that spans slots will not run, and a counter split across shards is not a counter.

Why not a distributed lock?

It works, and it is the wrong shape: a lock is two round trips plus a failure mode, where the script is one round trip and no failure mode.

Why the EXPIRE matters

Without it you keep one key per client forever. With it, an idle client costs nothing and its bucket comes back full — which is correct.

Production concerns

PART 6

The algorithm is a day of work. These are the decisions that decide whether the limiter helps you at 3 a.m. or becomes the outage.

Limiter is down

Fail open or fail closed?

Fail open for availability, but cap it: a local in-process limiter as the backstop.

Retry storm

Every client retries at the same second

Full jitter plus Retry-After. Without it the 429s themselves become the load.

One noisy tenant

Shared pool, one customer eats it

Per-tenant limits under the global one, and a reserved slice for everybody else.

Limit is a mystery

Clients cannot see where they are

RateLimit-* headers on every response, and the limit documented in numbers, not adjectives.

Figure 6.1 — The four failure modes worth naming unprompted. Each has one correct answer and one tempting wrong one.

6.2 Fail open or fail closed?

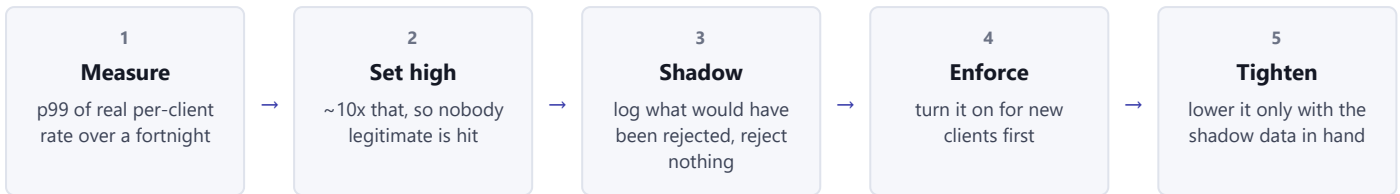
Fail open — the usual choice

Redis is down, so allow the request. You lose protection for the duration, but the API stays up. Pair it with a small in-process limiter so “open” still has a ceiling.

Fail closed — only when the limit is the product

Reject everything the limiter cannot verify. Correct for spend caps and hard quotas; catastrophic for a public API, where a Redis blip becomes a full outage.

6.3 Picking the number



The mistake is picking a round number and shipping it. The limit exists to stop the pathological case, so it should sit far above the legitimate one — and the only way to know where that is, is to have measured.

6.4 Things that are easy to get wrong

Counting rejected requests

A throttled client that keeps retrying should not dig its own hole deeper. Only record allowed requests — every implementation in this guide does.

Clock skew

Window boundaries computed from each node's own clock will not agree. Use the store's clock (Redis **TIME**) or a monotonic source plus one epoch.

Unbounded key growth

One key per client per window, with no TTL, is a memory leak with a schedule. Always **EXPIRE**.

Limiting after the expensive work

A limiter that runs after authentication, deserialisation and a database lookup has already spent what it was meant to save.

One config for all clients

Tiers are the whole business model: free, paid, partner, internal. Make the limit a lookup, not a constant.

No observability

Emit allowed, rejected and near-limit per client. The first question in an incident is "who is being throttled?" and you need it answerable in one query.

No way to raise a limit quickly

The limit will be wrong for somebody eventually. If changing it needs a deploy, you have built an incident. Keep it in config you can edit live.

Testing only the happy path

Test the boundary, the concurrent race, and the store being down. Those three are the entire bug surface of a rate limiter.

The interview itself

PART 7

“Design a rate limiter” is asked as an LLD question and as a component of half the HLD questions. The structure below fits in ninety seconds.

7.1 The order to say it in



THE NINETY-SECOND ANSWER

“I would put a token bucket at the API gateway keyed on the API key, because traffic is spiky and capacity gives me a burst allowance separate from the sustained rate. State goes in Redis behind a Lua script so refill and compare are one atomic round trip, with the client id in a hash tag so it stays on one slot. On rejection I return 429 with Retry-After, and I send RateLimit headers on successful responses too. If Redis is unreachable I fail open, but with a local in-process bucket as a backstop so ‘open’ still has a ceiling.”

7.2 What they are actually scoring

Did you name a trade-off?

Any of the five is a defensible answer. Picking one without saying what it costs is not.

Did you reach the distributed part?

Most candidates stop at the algorithm. The interesting half starts at the second app server.

Did you think about the caller?

429, Retry-After and jitter are half the design, and almost nobody volunteers them.

Did you handle your own failure?

A limiter on every request path is a new single point of failure. Say what happens when it breaks.

Q1 Design a rate limiter for our public API. Where do you start?

With what the limit protects, because that decides everything else. If it is capacity, it goes at the gateway keyed on the API key. If it is cost, it goes in the service and counts cost units. If it is abuse, it goes at the edge keyed on IP and it is deliberately blunt. Then I would name the four inputs — identity, limit, window, action — and only after that pick an algorithm.

Q2 Which algorithm, and why?

Token bucket for the general case: two numbers per client, and capacity lets me tolerate a burst without raising the sustained rate. If memory across millions of clients is the binding constraint I would use a sliding window counter instead, which is two integers and tracks an exact log to within a fraction of a percent. I would mention fixed window only to explain why I am not using it.

Q3 Explain the fixed window boundary problem.

The counter resets on a wall-clock boundary, not relative to the caller. With 100 per minute, a client takes 100 in the last second of one minute and 100 in the first second of the next — every window saw exactly 100, and the client got 200 in about two seconds. Sliding window fixes it by anchoring the window to now; buckets never had the problem because they never reset.

Q4 Token bucket and leaky bucket — aren't they the same thing?

In the counter form, yes — mathematically identical, with water = capacity minus tokens. Token bucket asks “do I have a token to spend?”, the leaky-bucket meter asks “is there room in the bucket?”, and those are the same comparison. The genuine difference is the queue form of leaky bucket, which holds requests and drains them at a fixed rate — that one reshapes the output, and it is the only algorithm here that can make a caller wait instead of telling it no.

Q5 How accurate is the sliding window counter, really?

It assumes the previous window's requests were spread evenly across it, so it is exact when traffic is smooth and drifts when it is not — under-counting if the previous window was back-loaded, over-counting if front-loaded. Cloudflare measured roughly 0.003% error on production traffic. I would say the number and then say the assumption, because the assumption is the part that can bite you.

Q6 You have fifty app servers. How does this still work?

A purely local limiter enforces fifty times the limit, so the state has to be shared. Central Redis with a Lua script is exact and costs one round trip per request. Local counters with an asynchronous sync are fast and approximate. I would default to Redis and only move to gossip if the round trip is measurably too expensive — and I would say out loud that gossip means the enforced limit is now a few percent loose.

Q7 Redis is on the hot path for every request. What happens when it dies?

I fail open, because a rate limiter taking down the API is worse than the API being briefly unprotected — but “open” still has a ceiling: each node falls back to a local in-process limiter sized at the global limit divided by node count. I would also circuit-break the Redis call so a slow Redis does not add its timeout to every request, and alert on the fallback being active. If the limit were a paid spend cap I would fail closed instead, and I would say why the answer flipped.

Q8 Two servers read the counter at 99 at the same instant. Walk me through it.

Both read 99, both compare against 100, both allow, and the counter lands at 101 — one request over, silently, and it gets worse with concurrency. The fix is to make the read and the write one operation: **INCR** is enough for a fixed window counter, and anything with more state — token bucket refill, sliding counter interpolation — needs a Lua script so the whole decision runs as one critical section on one shard.

Q9 What do you key the limit on?

The account or API key whenever the caller is authenticated, because that is the real identity and it survives the client changing networks. IP only for unauthenticated traffic, and knowing that carrier NAT puts thousands of users behind one address while IPv6 hands one user billions. In practice I would run several layers at once — a global ceiling, per-key, and a tighter one on the few endpoints that are expensive — and the narrowest layer is the one that usually says no.

Q10 What exactly do you return to the client?

429 Too Many Requests with **Retry-After**, plus **RateLimit-Limit**, **-Remaining** and **-Reset** — and I send those headers on successful responses too, so a well-behaved client can pace itself instead of discovering the limit by hitting it. Not 503, which means “we are overloaded”, and never a silent drop, which teaches the client to retry harder.

Q11 The client ignores it and keeps hammering. Now what?

First make sure I am not causing it: rejected requests must not extend the window, and **Retry-After** must be honest. Then escalate — a longer cooldown for repeat offenders, then a cheap deny at the edge so the request never reaches application code, and finally a block with a human in the loop. The important property is that rejecting must be far cheaper than serving, otherwise the abuse still wins.

Q12 How do you pick the actual number?

Measure the p99 of real per-client rate over a couple of weeks, set the limit around ten times that, run it in shadow mode logging what it would have rejected, then enforce — new clients first. Tighten only with shadow data in hand. The limit is there to stop the pathological caller, so it belongs far above the legitimate one, and picking a round number without measuring is how you page yourself.

Q13 Some requests cost a hundred times more than others. How do you limit that?

Count cost units rather than requests: tokens for an LLM call, rows scanned for a query, seconds of CPU. A token bucket takes this naturally — deduct N tokens instead of one. Where the cost is only known afterwards, reserve an estimate up front and reconcile the difference when the work finishes, which is exactly how metered APIs bill.

Q14 Gateway or service — where does the limiter belong?

Both, for different reasons. The gateway rejects cheaply and protects everything behind it, but it only knows the identity and the route. The service knows what the request will actually cost and can limit on that. A single limiter at the gateway is the right first answer; adding “and a cost-based one in the service for the expensive endpoints” is the senior one.

7.3 Red flags interviewers listen for

Saying this	Says this about you
Only naming one algorithm	Has memorised a blog post, not the trade-off
Fixed window presented as the final answer	Does not know the boundary bug
A local counter, with many app servers	Enforces N times the limit and has not noticed
GET then SET on a shared counter	Has never met a race condition
“Token bucket and leaky bucket are different”, with no detail	Repeating a comparison table without understanding it
No answer for the limiter being down	Has added a single point of failure to every request
Silently dropping over-limit requests	Guarantees a retry storm
Retries with no jitter	Will rebuild the exact spike they just shed
Keys with no TTL	A memory leak on a schedule
One limit for every customer	Has not thought about tiers, and cannot bill
Limiting after the expensive work	Has already spent what the limit was meant to save

7.4 Real systems worth name-dropping

Cloudflare’s sliding window counter

The blog post that made the two-counter approximation standard, with the measured error rate. The single best citation in this whole topic.

Stripe’s rate limiters

Four limiters of different kinds running together — request rate, concurrency, fleet usage, worker utilisation. Good evidence you think in layers.

Envoy / the gateway tier

Local and global rate limiting as a first-class filter, with a gRPC ratelimit service behind it. This is what “put it at the gateway” means in practice.

GCRA / the generic cell rate algorithm

A leaky bucket expressed as a single timestamp — no counter at all. Worth naming if you want to show you have read past the standard five.

One-page cheat sheet

The night-before page. If you remember only this, you can still hold a good conversation about rate limiting.

The five, in one line each

Algorithm	State	The one-line verdict
Fixed window	counter + window start	Cheapest. Leaks 2x across the boundary. Implement it, then criticise it.
Sliding log	a timestamp per allowed request	Exact and expensive. Use it when the number is billable.
Sliding counter	two counts + a bucket id	The production default: ~99.9% accurate for O(1) memory.
Token bucket	tokens + last-refill time	Burst up to capacity, then the refill rate. The usual right answer.
Leaky bucket, meter	water + last-leak time	Token bucket inside out. Same behaviour, different framing.
Leaky bucket, queue	the queue	The only one that delays instead of rejecting. Traffic shaping, not limiting.

The numbers

1M clients × 1,000 hits × 8 bytes = 8 GB of log · sliding counter error about 0.003% · N nodes local = N × the limit

Set the limit at roughly 10× the measured p99 per-client rate, always with a TTL on the key and always with jitter on the client's retry.

Rate limiting in eight facts

It is four decisions, not one

Identity, limit, window, action. The algorithm is the smallest of them.

Fixed window leaks 2x

The counter resets on the clock, not on the caller. That is the whole bug.

Sliding window anchors to now

No boundary means nothing to straddle. Log is exact; counter approximates it.

Token and leaky are the same

In counter form, $\text{water} = \text{capacity} - \text{tokens}$. Only the queue form differs.

Local counters multiply

N nodes each enforcing the limit enforce N times the limit.

Read-modify-write races

INCR for a counter, a Lua script for anything with more state. One shard, always.

Saying no has a cost

429 plus Retry-After plus jitter, or the rejections become the load.

Fail open, with a ceiling

The limiter must not be able to take down what it protects — unless it is the bill.

The five failure modes

Name	Mechanism	Primary fix
Boundary burst	Fixed window resets on wall-clock time	Sliding window counter, or a bucket
Limit multiplication	Every node counts independently	Shared state in Redis, or gossip with a shared budget
Counter race	GET then SET across nodes	INCR, or a Lua script keyed to one slot
Retry storm	Every rejected client retries on the same second	Retry-After plus full jitter; never drop silently
Limiter outage	Redis is unreachable and it is on every request path	Fail open behind a circuit breaker, with a local backstop limiter

IF YOU SAY NOTHING ELSE, SAY THIS

"A rate limit is four decisions — who I count, how many, over what window, and what I do when the answer is no. I would use a token bucket keyed on the API key at the gateway, because capacity gives me a burst allowance independent of the sustained rate. The state lives in Redis behind a Lua script so refill and compare are one atomic round trip on one slot, because otherwise fifty app servers each enforce the full limit and two of them will race on the same counter. Rejections get 429 with Retry-After, the RateLimit headers go on successful responses too, and clients retry with full jitter. If Redis is unreachable I fail open behind a local backstop — a rate limiter must never be the reason the API is down."

Sources & further reading — Cloudflare, *How we built rate limiting capable of scaling to millions of domains* (the sliding window counter and its measured error); Stripe engineering, *Scaling your API with rate limiters*; the Envoy documentation on local and global rate limiting; the IETF draft on the **RateLimit-Limit**, **-Remaining** and **-Reset** header fields; RFC 6585 §4 for 429; AWS Architecture Blog, *Exponential backoff and jitter*; the ITU-T and ATM Forum definitions of the generic cell rate algorithm (GCRA); the Redis documentation on Lua scripting, hash tags and **INCR/EXPIRE**; Kong, NGINX and Envoy rate-limit module documentation for how these ship in practice.